Full Lazard CAD Using Rational Polynomials and Triangular Real-Point Certificates

Executable pseudocode, correctness proofs, and audit requirements

CLPoly Project

October 22, 2026

Abstract

All stored polynomials in this specification have rational coefficients. A real algebraic point is specified by a squarefree regular triangular chain and a certified isolating box, following Xia–Zhang (2006). No instruction substitutes an irrational number into a polynomial object. Specialization is represented by keeping the rational polynomial together with the point certificate. We give terminating algorithms for selected-point zero tests and signs, coefficientwise zero tests, and Lazard evaluation, with proofs. Triangular decomposition routines are imported explicitly from Xia–Yang (2016); isolation and refinement follow Xia–Zhang (2006). Xia–Yang (2002) is supplementary only. The CAD implementation uses this rational-data representation. Complete lifting is specified and proved relative to the explicitly imported projection theorem below. These are mathematical proofs, not machine-checked verification of C++.

Contents

1 Authority, scope, and permitted primitives **2**

2 Representation and invariants **3**

3 XZ06 isolation and refinement, made operational **4**
3.1 Interval bounds and positive shifts 4
3.2 NSHR and arbitrary requested precision 5

4 Selected-point zero and sign without extension arithmetic **7**

5 Executable coefficientwise specialization test **8**

6 Lazard definition, full pseudocode, and proof **9**

7 Output use in lifting: branch restriction before root isolation **10**

8 UNI-QQ: the checked rational univariate adapter **13**

9 Complete CAD: projection contract and cell semantics **14**
9.1 Projection data and the imported theorem 15
9.2 The meaning of stored nodes 15

10 Complete CAD: executable lifting and driver **16**
10.1 Simplest rational samples in open intervals 16
10.2 Loop invariants and implementation correspondence 20
10.3 Correctness and termination 21

11 Examples and mandatory regressions **22**

12 Implementation audit and formal-proof boundary **22**

13 Checked interfaces and lexicographic implementation **24**

1 Authority, scope, and permitted primitives

This document specifies real-point operations, QQ Lazard evaluation, and complete CAD lifting. The projection theorem is imported explicitly from the baseline research and MPP; lifting, cell semantics and the driver are proved relative to that theorem. Runtime polynomials always have rational coefficients.

The following cited algorithms are permitted subroutines; they are not unspecified algebraic-number oracles. Their inputs, outputs, and restrictions are part of the contract. All calls here have *no free parameters* ($K = \mathbb{Q}$, $u = \varnothing$).

Primitive	Exact source	Contract used here
WRSD	XY16 Algorithm 2.2, pp.40–41; Theorem 2.6	Returns (H, G) with $V(T, c) = \bigcup_{C \in H} V(C)$ and $V(T) \setminus V(c) = \bigcup_{C \in G} V(C)$.
RCtoSqrfree	XY16 Algorithm 3.1 and Theorem 3.2, p.62	Finite squarefree regular decomposition of a zero-dimensional regular chain; includes the first polynomial, not only higher levels.
NREALZERO	XZ06 §3, pp.855–858	Isolation of every real solution of a regular TSA, after the positive-root shifts on p.856. Retain bound-polynomial and monotonicity certificates.
NSHR	XZ06 pp.857–858	Refine a certified box from NREALZERO without changing its selected solution. It is not an independent refinement of arbitrary coordinate intervals.
UNI	Rational univariate isolation, as permitted in XZ06 §3	All real roots of a nonzero squarefree rational polynomial, with positive precision, in sorted strict brackets. Algorithm 11 clears positive denominators and supplies zero explicitly around the trusted nonzero-root isolator.

We use polynomial arithmetic, rational comparisons, differentiation, coefficient collection, and rational interval arithmetic as elementary primitives. No inverse in a quotient algebra and no gcd over an algebraic coefficient field is needed. XY02 REALZERO may report failure; it is not substituted for NREALZERO. The legacy RSD implementation and its existing test remain available. The new algorithms do not invoke it, directly or through their current `charset.hh` call paths; RSD is not an imported primitive here. The nonreduced-prefix case $T = [x^2, y^3]$, $c = xy$ prevents treating the legacy backend as

terminating on every regular input. The project note `docs/cad/fixes/rsd-nonradical-prefix.md` records the counterexample and the scope of the unresolved literature conditions. The squarefree decomposition contract above remains a required interface. The current C++ backend can repeat the same input on $[x^2, y^3, z^2 + xy]$: a valid WRSD zero branch need not simplify the prefix. Until this progress issue is repaired, implementation conformance for arbitrary regular-chain inputs is not established. This does not affect the proof below that POINT-ZERO itself uses only the WRSD set contract.

2 Representation and invariants

Fix the *lifting* coordinate order $x_1, \ldots, x_k$. The public CAD argument is instead the *projection* order. For a public tuple $V = (v_1, \ldots, v_n)$, the local lifting notation means $x_i = v_{n+1-i}$: projection eliminates $v_1, v_2, \ldots$, whereas lifting first isolates v_n and then extends towards v_1. The C++ triangular-chain constructor receives the lifting tuple and a pointer to an external lexicographic comparator. The tuple must be the reversed contiguous suffix of the comparator's declared variable-priority list; this also preserves the consecutive class indices used by WRSD. The main variable is always the first, or smallest, variable under that comparator; the chain equations and rational box remain in lifting order.

Definition 2.1 (Valid real-point certificate)**.** A certificate $A = (T, B, \mathcal{C})$ consists of rational polynomials $T = [t_1(x_1), \ldots, t_k(x_1, \ldots, x_k)]$, a rational box $B = \prod_i [a_i, b_i]$, and the isolation/refinement evidence $\mathcal{C}$. Each t_i has positive degree in x_i. The chain is zero-dimensional, regular and squarefree: on *every complex zero* of its preceding prefix, $\operatorname{lc}(t_i, x_i) \operatorname{disc}(t_i, x_i)$ is nonzero; t_1 has no multiple root. Every prefix box contains exactly one real zero of the corresponding prefix chain. The selected point is in the interior of its box. Evidence $\mathcal{C}$ records the rational coordinate shifts and the paired bound-root certificates used by NREALZERO/NSHR. Rational roots can always be enclosed in strict rational brackets. For $k = 0$, T and B are empty and designate the unique empty point.

An arbitrary box containing one solution is not automatically an NSHR certificate. Constructor validation must establish the stated conditions or obtain a certificate from NREALZERO. Rational coordinates may be represented by $q_i x_i - p_i = 0$; they are processed in exactly the same order as the other coordinates. Each C++ triangular-chain object stores only a pointer to an external comparator. Default copy and move operations preserve this pointer together with the polynomial and monomial comparator pointers. Prefixes and decomposition branches borrow the same comparator without rebinding or normalization. The caller must keep it alive and its variable order unchanged for all chains, points and converted polynomials that borrow it. The C++ extension routines `append_rational` and `roots_over` receive this pointer explicitly; no returned point borrows a comparator local to these routines.

A stored polynomial representative is always

$$p \in \mathbb{Q}[x_1, \ldots, x_k, y].$$

For a prefix length i, define the *semantic* map

$$E_i(p) = p(\alpha_1, \ldots, \alpha_i, x_{i+1}, \ldots, x_k, y).$$

E_i is used in specifications and proofs only. It is never an executable instruction. The representation (p, A, i) has this meaning, while its data remains rational. Variables $x_1, \ldots, x_i$ are frozen representative variables; the remaining ones are active. A prefix certificate uses the restricted variable order, not an order containing undeclared later variables.

3 XZ06 isolation and refinement, made operational

3.1 Interval bounds and positive shifts

For $q(u,z) = \sum_{j=0}^{d} q_j(u) z^j$ and a rational prefix box X, compute $[c_j, d_j] = \mathrm{IE}(q_j, X)$ and set

$$L(z) = \sum c_j z^j, \qquad U(z) = \sum d_j z^j.$$

XZ06 Proposition 3.1 gives $L^{(r)}(z) \le q^{(r)}(\alpha, z) \le U^{(r)}(z)$ for $z > 0$ and all $r \ge 0$. The same inequalities hold for all $u \in X$. The bounds are rational polynomials, not necessarily integer polynomials.

To make the shift on XZ06 p.856 executable, first refine until $0 \notin \mathrm{IE}(q_d, X)$. Let $m = \min(|c_d|, |d_d|) > 0$ and $M = \max_{j<d} \max(|c_j|, |d_j|)$. Choose rational

$$R > 1 + M/m, \qquad \widehat{q}(u,z) = q(u, z - R).$$

Every real root in the old coordinate y satisfies $|y| < R$, so its new coordinate $z = y + R$ is strictly positive. This includes negative and zero roots. The shift and its inverse must remain in the certificate. This is a concrete root-bound choice for XZ06's shift, not a different isolation algorithm. Indeed, for $|y| > 1 + M/m$, the magnitude of the leading term exceeds the sum of all lower-term magnitudes by the geometric-series bound, so $q(u,y) \neq 0$.

Algorithm 1 XZ-LIFT: certified version of NREALZERO Step 2

Require: Valid prefix certificate $A = (T, X, \mathcal{C})$; q with nonvanishing leading coefficient and simple roots at the selected prefix

Ensure: All distinct roots of $q(\alpha, y)$, represented by q, A and strict intervals

Determine and record the shift R above; set $\widehat{q}(u,z) = q(u, z-R)$
$n \leftarrow 0$
loop
  $(L, U) \leftarrow$ coefficient interval bounds of $\widehat{q}$ on X
  $n \leftarrow n + 1$
  if L, U do not have degree d, equal leading signs, or are not squarefree **then**
    $X \leftarrow \mathrm{NSHR}(A)$; **continue**
  end if
  $S_L \leftarrow \mathrm{UNI}(L, 2^{-n})$; $S_U \leftarrow \mathrm{UNI}(U, 2^{-n})$
  if either list has a nonpositive root or their lengths differ **then**
    $X \leftarrow \mathrm{NSHR}(A)$; **continue**
  end if
  Pair roots by increasing order; let J_j be the hull of their strict brackets
  if the J_j are positive and disjoint, and L', U' have the same strict sign on each J_j **then**
    return $(q, A, \{J_j - R\}_j)$ and bound/shift certificates
  end if
  $X \leftarrow \mathrm{NSHR}(A)$
end loop

The derivative acceptance tests use rational interval evaluation of L' and U'. For empty prefixes NSHR is a no-op; univariate bracket precision still increases. A constant nonzero q returns the empty list before calling this algorithm. A zero specialization is a separate error, never an empty finite root set. The additional squarefree and strict-sign checks are explicit sufficient certificates for XZ06's pairing/monotonicity conditions; they are not claimed to be verbatim paper lines.

Lemma 3.1 (XZ-LIFT correctness and termination)**.** *Algorithm 1 returns exactly all roots, without requiring irrational coefficient operations.*

Proof. At acceptance, both bounding polynomials have the same number of simple real roots, all positive, the same degree and leading sign. Outside the paired hulls, they have the same nonzero sign: count crossings from $+\infty$, one simple root from each polynomial per hull. Thus the target polynomial cannot vanish outside these hulls. On each hull the common strict derivative sign implies strict monotonicity of the target, by XZ06 Proposition 3.1. At the two endpoints both bounds have opposite signs, hence so does the target. The intermediate value theorem gives exactly one root. Shifting back is a bijection.

As the prefix shrinks to its selected point, coefficient interval widths tend to zero. Both bound polynomials converge to the same fixed polynomial of degree d with simple roots, none at zero. In a sufficiently small coefficient neighborhood, each simple real root persists and stays separated, and each nonreal root remains nonreal. Their ordered real roots therefore pair and converge. The univariate bracket widths also tend to zero. Derivatives at the limiting roots are nonzero, so the strict interval derivative certificates eventually hold. All acceptance conditions are eventually met. This is the stability argument underlying XZ06 §3, with explicit sufficient acceptance predicates. □

3.2 NSHR and arbitrary requested precision

For review, the imported NSHR steps are: shrink the first-coordinate bracket to at most half its width by rational univariate bracketing; for each subsequent coordinate, compute new lower and upper bound polynomials using the already refined prefix; bracket their respective unique roots inside the old coordinate interval to width at most 1/8 of that interval; take the hull of these two brackets. Retain the same shift and equation. The preceding strict derivative certificates and monotonicity of interval evaluation preserve the required roots and monotonicity on every later call.

Univariate bracketing is finite rational bisection of a unique simple root with opposite endpoint signs. If the midpoint is exactly a root, keep strict endpoints on its two sides rather than returning a degenerate bracket. Such endpoints may be chosen arbitrarily close to the rational midpoint; the established monotonicity gives their signs. Thus no point interval is required.

Algorithm 2 NSHR-QQ: explicit rational-data transcription of XZ06 NSHR

Require: Valid certificate $A = (T, B, \mathcal{C})$ with its per-level bound/shift evidence
Ensure: Nested valid box selecting the same point

if $T = []$ **then**
  return A
end if
Save old box B^{old}
$B_1 \leftarrow \mathrm{BRACKET}(t_1, B_1^{old}, \mathrm{width}(B_1^{old})/2)$
for $j = 2, \ldots, k$ **do**
  $R_j \leftarrow$ recorded shift for level j
  $q(u, z) \leftarrow t_j(u, z - R_j)$ by rational polynomial translation
  $(L, U) \leftarrow$ coefficient interval bounds of q on the updated prefix $B_{<j}$
  $J \leftarrow B_j^{old} + R_j$; $\epsilon \leftarrow \mathrm{width}(B_j^{old})/8$
  $I_L \leftarrow \mathrm{BRACKET}(L, J, \epsilon)$; $I_U \leftarrow \mathrm{BRACKET}(U, J, \epsilon)$
  $B_j \leftarrow \mathrm{hull}(I_L, I_U) - R_j$; update bound evidence
end for
return updated A

BRACKET is the strict rational bisection procedure described above: while width exceeds the bound, evaluate the midpoint rationally and retain the opposite-sign half; for an exact midpoint root choose strict brackets on its two sides of width at most half the old width and at most the requested width. The input polynomial is strictly monotone on the input interval and has opposite endpoint

signs, so every step is defined. For level one a univariate root certificate supplies the same unique-root bracketing property; monotonicity can be certified during construction by refining its simple-root interval.

Algorithm 3 REFINE: precision wrapper around XZ06 NSHR

Require: Valid A, rational $\epsilon > 0$
Ensure: Same selected point, nested certified box, every width $< \epsilon$
1: **while** $\max_j \text{width}(B_j) \geq \epsilon$ **do**
2: $A \leftarrow \text{NSHR}(A)$ ▷ XZ06 pp.857–858; empty point needs no loop
3: **end while**
4: **return** A

Lemma 3.2 (REFINE terminates)**.** *Successive NSHR calls preserve the point and make every coordinate width tend to zero.*

Proof. Point preservation is XZ06's NSHR argument: the updated bounds lie between the old bounds, and their unique roots bracket the target root. Their hull is inside the old interval. The first width decreases by a factor at most $1/2$. Inductively suppose all earlier widths tend to zero. The two bounding roots at level j converge to the same simple target root, so their distance δ_n tends to zero. Since each bracket width is at most $w_{j,n}/8$, the next hull satisfies $w_{j,n+1} \leq \delta_n + w_{j,n}/4$. These nonnegative widths are nested; taking limits gives $w_j \leq w_j/4$, hence $w_j = 0$. Induction proves the claim. Do not replace this argument by the false assertion that every NSHR call halves *every* coordinate width. □

Algorithm 4 ISOLATE-REGULAR: equation-only NREALZERO driver

Require: A squarefree zero-dimensional regular rational chain T in the stated order
Ensure: Certificates for all distinct real solutions of T
1: **if** $T = []$ **then**
2: **return** the singleton list containing the empty point
3: **end if**
4: $L \leftarrow \text{UNI}(t_1)$; refine its brackets until $0 \notin \text{IE}(t_1', J)$
5: Attach $[t_1]$ and the strict root evidence to each interval in L
6: **for** $j = 2, \ldots, k$ **do**
7: $L_{new} \leftarrow []$
8: **for** each prefix certificate A in L **do**
9: $S \leftarrow \text{XZ-LIFT}(t_j, A)$
10: Append every extended certificate (A, t_j, J), $J \in S$, to L_{new}
11: **end for**
12: $L \leftarrow L_{new}$
13: **end for**
14: **return** L

This is NREALZERO for the pure-equation case needed here. Inequality filtering (Step 3 in the paper) is not invoked. The first coordinate is isolated on the full real line by UNI; later levels use the recorded positive shifts. The free functions `shrink` and `refine` implement Algorithms 2 and 3; the point member functions only forward to them. The NSHR body works in original coordinates with a shift only for the active bound polynomial. By induction on j, Lemma 3.1 gives all extensions of each prefix and no others. Distinct prefixes or distinct intervals within a fibre give distinct points. The regular squarefree hypothesis establishes every XZ-LIFT precondition.

4 Selected-point zero and sign without extension arithmetic

WRSD directly returns two families (Z, N) with

$$\bigcup_{C\in Z} V(C) = V(T)\cap V(c), \qquad \bigcup_{C\in N} V(C) = V(T)\setminus V(c).$$

These set equalities are the entire decomposition contract used below. The zero label does not require $\mathrm{prem}(c; C) = 0$ and is never reconstructed from a pseudo-remainder test. Branches need not be pairwise disjoint and need not themselves be squarefree: we refine only the original valid certificate, never a newly returned branch box.

Algorithm 5 POINT-ZERO: exact zero at one selected branch

Require: Valid $A = (T, B, \mathcal{C})$ of dimension i; $c \in \mathbb{Q}[x_1, \ldots, x_i]$
Ensure: Returns true exactly when $c(\alpha) = 0$; may refine B

if $i = 0$ **then**
  return $(c = 0)$ by rational comparison
end if
$(Z, N) \leftarrow \mathrm{WRSD}(T, c)$
loop
  Remove from Z and N each C with some $g \in C$ satisfying $0 \notin \mathrm{IE}(g, B)$
  require $Z \cup N \neq \varnothing$
  if $N = \varnothing$ **then**
    return true
  end if
  if $Z = \varnothing$ **then**
    return false
  end if
  $A \leftarrow \mathrm{REFINE}(A, \frac{1}{2}\max_j \mathrm{width}(B_j))$
end loop

All equations in C are evaluated on the original coordinate box, not on separate univariate resultant-root candidates. A component is removed only by a certified nonzero equation; interval overlap alone never proves membership.

Theorem 4.1 (POINT-ZERO is a decision procedure)**.** *Algorithm 5 terminates and its Boolean result is correct.*

Proof. WRSD terminates and supplies finitely many regular components with the two set equalities stated above. A zero-labelled component consists entirely of zeros of c, and a nonzero-labelled component has no zero of c. Thus no point lies in components with opposite labels. No assertion about membership in a nonradical saturated ideal is needed.

At least one component contains α. None of its equations can have an interval enclosure excluding zero, so it is never removed. Every component not containing α has an equation g with $g(\alpha) \neq 0$. Polynomial interval evaluation is inclusion preserving and converges on shrinking boxes; hence that component is eventually removed. There are finitely many components. Eventually all remaining components contain α and have the same label, forcing the correct return. REFINE terminates by Lemma 3.2. □

Algorithm 6 POINT-SIGN

Require: Valid A; rational polynomial c in its coordinates
Ensure: Exact sign of $c(\alpha)$

$[l, u] \leftarrow \mathrm{IE}(c, B)$
if $l > 0$ **then**
    return $+1$
end if
if $u < 0$ **then**
    return -1
end if
if POINT-ZERO(c, A) **then**
    return 0
end if
loop
    $[l, u] \leftarrow \mathrm{IE}(c, B)$
    if $l > 0$ **then**
        return $+1$
    end if
    if $u < 0$ **then**
        return -1
    end if
    $A \leftarrow \mathrm{REFINE}(A, \frac{1}{2} \max_j \mathrm{width}(B_j))$
end loop

For dimension zero the interval is the exact rational singleton and a nonzero constant returns before refinement. Correctness follows from inclusion; the initial strict-sign path is immediate, and the remaining path terminates by the exact nonzero decision and interval convergence. An interval such as $[0, b]$, $b > 0$, proves neither zero nor a strict sign.

5 Executable coefficientwise specialization test

Algorithm 7 PREFIX-ZERO: no irrational substitution

Require: $p \in \mathbb{Q}[x_1, \ldots, x_k, y]$; valid prefix A_i; active variables $W = (x_{i+1}, \ldots, x_k, y)$
Ensure: Returns true exactly when $E_i(p)$ is the zero polynomial

$M \leftarrow$ empty coefficient map (missing entries mean zero)
for each term $ax^\beta W^\gamma$ of p **do**
    $M[\gamma] \leftarrow M[\gamma] + ax^\beta$ ▷ collect equal active monomials first
end for
for each nonzero collected coefficient c in M **do**
    if not POINT-ZERO(c, A_i) **then**
        return false
    end if
end for
return true

Lemma 5.1. *PREFIX-ZERO terminates and has its stated meaning.*

Proof. Collection is a finite rational polynomial computation and yields $p = \sum_\gamma c_\gamma(x_1, \ldots, x_i) W^\gamma$.

Distinct monomials in the still formal variables W are linearly independent. Consequently $E_i(p) = 0$ if and only if every $c_\gamma(\alpha) = 0$. Apply Theorem 4.1 to the finitely many coefficients. □

6 Lazard definition, full pseudocode, and proof

For a nonzero rational polynomial f, define its Taylor expansion semantically by

$$f = \sum_{\beta \in \mathbb{N}^k} c_\beta(y) \prod_{j=1}^{k} (x_j - \alpha_j)^{\beta_j}.$$

The support $S = \{\beta : c_\beta \neq 0\}$ is finite and nonempty because translation is invertible. The valuation is $v = \min_{\text{lex}} S$ with x_1 highest priority; the Lazard value is $f_\alpha = c_v(y)$. This defines the result; no Taylor expansion with irrational coefficients is constructed.

Algorithm 8 LAZARD-QQ: frozen rational representatives

Require: Nonzero $f \in \mathbb{Q}[x_1, \ldots, x_k, y]$; valid A; this explicit coordinate order
Ensure: Return (p, v), interpreted relative to the unchanged input point A, with $E_k(p) = f_\alpha \neq 0$ and $v = v_\alpha(f)$

1: $p \leftarrow f$; $v \leftarrow []$
2: **for** $i = 1, \ldots, k$ **do**
3: $A_i \leftarrow$ certified prefix of A in its restricted variable order
4: $h \leftarrow p$; $r \leftarrow 0$; $b \leftarrow 1$
5: **while** PREFIX-ZERO$(h, A_i, (x_{i+1}, \ldots, x_k, y))$ **do**
6: $h \leftarrow \partial h / \partial x_i$ ▷ differentiate the unreduced rational representative
7: $r \leftarrow r + 1$; $b \leftarrow br$
8: **end while**
9: $p \leftarrow h/b$; append r to v ▷ division only by a nonzero integer
10: **end for**
11: **return** (p, v)

Reject zero input before line 1. For $k = 0$ return $(f, [])$. The caller retains A; the returned C++ object owns only its QQ representative, variable comparator and valuation. It is not a self-contained algebraic value, and must not be reinterpreted using a different conjugate point. Prefix refinements may be kept as local copies or propagated to the parent box; coordinate nesting preserves the point either way. No equation of T is used to reduce p or h. In particular, the algorithm neither divides by $x_i - \alpha_i$ nor drops a pseudo-division multiplier. Optional future reductions require a separate proof that they commute with all remaining derivatives.

Lemma 6.1 (Derivative extraction)**.** *For a nonzero $g(z) = \sum_{j=0}^{d} a_j (z-\theta)^j$ over a polynomial ring with real coefficients, the first nonzero derivative specialized at $z = \theta$ is $g^{(r)}(\theta) = r! a_r$, where r is the first nonzero coefficient index.*

Proof. Differentiating r times and setting $z = \theta$ kills every translated monomial except exponent r, whose multiplier is $r!$. Some coefficient is nonzero and factorials are invertible in the real coefficient ring used here. □

Theorem 6.2 (LAZARD-QQ total correctness)**.** *Algorithm 8 terminates and meets its postcondition. Every computed polynomial has rational coefficients.*

Proof. After iteration i the invariant is

$$E_i(p) = \sum_{\substack{\beta \in S \\ \beta_1 = v_1, \ldots, \beta_i = v_i}} c_\beta(y) \prod_{j>i} (x_j - \alpha_j)^{\beta_j} \neq 0.$$

It holds initially with the entire Taylor expansion. At iteration i, earlier specialization commutes with differentiation in the distinct variable x_i: $E_{i-1}(\partial^r_{x_i} p) = \partial^r_{x_i} E_{i-1}(p)$. PREFIX-ZERO tests exactly whether the next specialization of this derivative is zero as a polynomial in the later variables. Derivative extraction therefore selects the smallest possible exponent v_i and division by $v_i!$ gives the asserted invariant. Keeping the earlier variables in the representative does not change this semantic identity.

Successive smallest-prefix choices are exactly lexicographic minimization. At $i = k$ the invariant is $E_k(p) = c_v(y) \neq 0$. Each inner loop has at most $\deg_{x_i}(p)$ differentiations, since the nonzero semantic polynomial has no larger degree than its representative. Every zero test terminates. The outer loop has k steps, so the algorithm terminates. Differentiation, coefficient grouping, and division by factorials preserve rational coefficients. All interval and triangular subroutines also operate on rational data. □

7 Output use in lifting: branch restriction before root isolation

Lifting interprets each QQ representative at the selected prefix point. Branch restriction preserves the zero-dimensional hypotheses required by the cited triangular isolation algorithms.

A point belongs to a candidate component C exactly when POINT-ZERO returns true for every $g \in C$. This terminates and works even when components overlap. The selected prefix is re-encoded recursively, then only its next coordinate is lifted. No interval-overlap membership test is used.

Write LIFT-ONE(C, P) for one execution of NREALZERO Step 2: C is a squarefree regular chain of length k, and P is a valid certificate on its length-$k-1$ prefix. Call XZ-LIFT on the last equation at P, and append each returned strict root bracket and its fixed shift to the refined prefix. The inherited prefix certificates and the new XZ-LIFT evidence certify the combined point, so no already verified prefix equation needs to be re-certified. This helper is implemented by `__lift_real_points`; full-chain ISOLATE-REGULAR calls it on every prefix, while ROOTS-QQ calls it only on its selected prefix.

Algorithm 9 REENCODE: retain the selected point on another regular chain

Require: Valid $A = (T, B, \mathcal{C})$; regular chain C on the same k coordinates
Ensure: For $\alpha \in V(C)$, a squarefree certificate for α; otherwise reject

if $C = T$ as normalized equation lists **then**
  return A
end if
Require POINT-ZERO to hold on A for every equation of C
if $k = 0$ **then**
  return A
end if
for $D \in$ RCtoSqrfree(C) **do**
  if C was not squarefree and some equation of D fails POINT-ZERO at A **then**
    continue
  end if
  $P \leftarrow$ REENCODE$(A_{<k}, D_{<k})$
  for $Q \in$ LIFT-ONE(D, P) **do**
    if SAME-POINT(Q, A) **then**
      return Q
    end if
  end for
end for
Report loss of the selected point

SAME-POINT first tests every equation of the second chain at the first point using POINT-ZERO. If any test fails, the points differ. Otherwise refine the first certificate until its box is contained in the interior of the second box (equal) or is disjoint from it (different). The first point then lies on the second chain, whose certificate has one interior zero and none on its boundary. This proves correctness and termination of the comparison, without assuming that overlapping boxes represent the same point.

Lemma 7.1 (Selected-prefix re-encoding)**.** *REENCODE terminates and returns the same point on a squarefree component of C.*

Proof. Induct on k. Identical normalized chains preserve the existing certificate; $k = 0$ is the empty point. Otherwise the initial membership tests verify the precondition, and RCtoSqrfree supplies finitely many covering squarefree chains. At least one contains α; the branch tests remove only those that do not. By induction, its recursive call gives exactly the first $k-1$ coordinates of α. XZ-LIFT then enumerates all last-coordinate roots on this prefix, including α_k. SAME-POINT accepts exactly the desired full point. All branch lists and root lists are finite, and recursion lowers k. No global containment $V(C) \subseteq V(T)$ is needed because candidate membership is tested before box matching. Independent chain/point prefix operations only restrict already verified certificates and preserve the external comparator pointer. □

Algorithm 10 ROOTS-QQ: isolate roots only over the selected prefix

Require: Valid $A = (T, B, \mathcal{C})$; $p = \sum_{j=0}^{d} c_j(x)y^j$
Ensure: For $E_k(p) \neq 0$, all distinct fibre roots; reject an identically zero fibre

$C \leftarrow T$; $e \leftarrow -1$
for $j = d, d-1, \ldots, 0$ **do**
    $(H, G) \leftarrow \mathrm{WRSD}(C, c_j)$
    Choose $D \in H \cup G$ containing α by POINT-ZERO on its equations
    $C \leftarrow D$
    if $D \in G$ **then**
        $e \leftarrow j$; **break**
    end if
end for
if $e < 0$ **then**
    Reject the identically zero specialization
end if
if $e = 0$ **then**
    return empty list
end if
$q \leftarrow \sum_{j=0}^{e} c_j y^j$; $R \leftarrow []$
for $D \in \mathrm{RCtoSqrfree}([C, q])$ **do**
    if some equation of $D_{\leq k}$ fails POINT-ZERO at A **then**
        continue
    end if
    $P \leftarrow \mathrm{REENCODE}(A, D_{\leq k})$
    for $Q \in \mathrm{LIFT\text{-}ONE}(D, P)$ **do**
        duplicate $\leftarrow$ false
        for $Q' \in R$ **do**
            if their last-coordinate brackets are disjoint **then**
                continue
            end if
            if $\mathrm{SAME\text{-}POINT}(Q, Q')$ **then**
                duplicate $\leftarrow$ true; **break**
            end if
            while their last-coordinate brackets overlap **do**
                Refine Q and Q' to half their respective maximum widths
            end while
        end for
        if not *duplicate* **then**
            Append Q to R
        end if
    end for
end for
Sort R by the left endpoints of its disjoint last brackets; **return** R

For an empty parent, coefficient labels are rational comparisons; no triangular routine is called on an empty chain. LIFT-ONE uses XZ-LIFT with the empty prefix and the rational UNI adapter, so this case is included in the same control flow.

Lemma 7.2 (ROOTS-QQ contract)**.** *All calls in Algorithm 10 satisfy their algebraic preconditions, and*

its result is complete and exact.

Proof. WRSD covers the current chain and partitions its zeros by each coefficient's vanishing. The selected point persists. On the selected chain all skipped higher coefficients vanish at every zero, and the first nonzero-labelled c_e is nonzero at every zero. If no such coefficient exists the fibre is identically zero; if $e = 0$ it has no roots. Otherwise $[C, q]$ is zero-dimensional and regular. RCtoSqrfree preserves its zeros and ensures all LIFT-ONE simple-root hypotheses. A branch whose prefix does not contain α contributes no target fibre root and can be discarded before isolation. On each retained branch, REENCODE preserves the selected prefix and XZ-LIFT returns all last-coordinate roots on that prefix, so the union is exactly the target root set. There is no need to isolate real points over other prefixes.

All candidates have the same first k coordinates. Disjoint last brackets therefore prove inequality; overlapping ones use the exact SAME-POINT test. Only equal points are removed. For distinct last coordinates, convergence and positive separation make the persistent refinements terminate. After each insertion all stored last brackets are pairwise disjoint; later refinements are nested and preserve that invariant. Sorting by their left endpoints is a strict order comparison with no algebraic work inside the comparator. Each list and loop is finite, and every invoked decision/refinement terminates. □

This construction is implemented by `roots_over`; its correctness is relative to the same XY16 and XZ06 contracts as full-chain isolation. Taking $T \cup \{p\}$ without first restricting the degree branch can produce a positive-dimensional system and is forbidden.

8 UNI-QQ: the checked rational univariate adapter

The scalar integer `uspensky` interface inherited from the trusted baseline returns the *nonzero* real roots of a squarefree polynomial. It is not by itself the UNI contract used here. The following adapter supplies zero explicitly. UNI in Algorithms 1–10 means this checked adapter; its input must be squarefree and its requested precision positive. A nonzero constant is allowed. Zero and nonsquarefree inputs are rejected, not silently interpreted as finite empty root sets. All XZ-LIFT callers check squarefreeness before UNI, and ISOLATE-REGULAR has a squarefree chain as its precondition.

Algorithm 11 UNI-QQ: all simple real roots with strict brackets

Require: Rational coefficient vector f; rational precision ϵ

Ensure: For squarefree nonzero f and $\epsilon > 0$, all real roots in sorted, strictly separated brackets of positive width at most ϵ, with strict derivative sign

Reject $\epsilon \le 0$; clear a common positive denominator to obtain $z \in \mathbb{Z}[t]$
Reject $z = 0$; if $\deg z = 0$, return []
Reject unless $\deg \gcd(z, z') = 0$
$L \leftarrow$ ScalarUspensky(z) ▷ trusted nonzero-root interface
if $f(0) = 0$ **then**
    append $[0, 0]$ to L
end if
for each interval $J \in L$ **do**
    if an endpoint r of J is an exact root **then**
        $\delta \leftarrow \epsilon/4$
        repeat
            $J \leftarrow [r - \delta, r + \delta]$; $\delta \leftarrow \delta/2$
        until IE(f', J) has a strict sign and $f(\inf J) f(\sup J) < 0$
    end if
    $J \leftarrow$ BRACKET(f, J, ϵ)
    while IE(f', J) has no strict sign **do**
        $J \leftarrow$ BRACKET$(f, J, \text{width}(J)/2)$
    end while
end for
Sort L by left endpoints
for each adjacent pair J, K in L **do**
    while $J \cap K \neq \varnothing$ **do**
        Replace each by its BRACKET refinement to half its width
    end while
end for
return L

Lemma 8.1 (UNI adapter correctness and termination)**.** *Algorithm 11 satisfies its checked contract.*

Proof. Positive denominator clearing preserves the roots. The gcd test ensures every root is simple. ScalarUspensky supplies each nonzero root exactly once, so the additional zero singleton supplies precisely the missing root, without duplicates. At every simple root the derivative is nonzero; hence sufficiently small rational neighborhoods have strict derivative sign and opposite endpoint signs. This proves termination and validity of the endpoint conversion. A strict isolating bracket of a simple root can be bisected even before a derivative sign has been certified: the unique root is retained by opposite endpoint signs. If the midpoint equals that root, the chosen smaller symmetric bracket lies inside the old one and contains no other root. Subsequent bisection makes the derivative interval converge to its nonzero value. Distinct roots have positive separation, so neighboring brackets eventually become disjoint. Later refinements are nested and cannot destroy earlier separations. Every operation is rational and finite. □

9 Complete CAD: projection contract and cell semantics

This section closes the lifting proof using the projection/delineability theorem already referenced in the baseline project research. It does not replace XY16 or XZ06 with another algebraic-number algorithm.

We explicitly import the mathematical result of McCallum–Parusiński–Paunescu (MPP)[1], Theorem 5.1 and §5.3, and the trusted baseline projection, factorization and tree-storage contracts. The following is an implementation-level refinement of MPP Algorithm 2.12. It is a mathematical proof relative to these imports, not a machine proof of them.

9.1 Projection data and the imported theorem

Write $x_1, \ldots, x_n$ for the *lifting* order; the public API receives the reverse *projection* order $(x_n, \ldots, x_1)$. Set A_n to the input after clearing positive rational denominators, retaining the original rational inputs separately. At stage k, the trusted FACTOR projection supplies B_k, the pairwise nonassociate primitive irreducible factors of positive degree in x_k, and passes to A_{k-1} the contents/lower-variable factors and the nonconstant leading coefficients, trailing coefficients, discriminants and pairwise resultants of B_k. Zero input polynomials are discarded for projection and retained for final signs; nonzero constants need no projection. Thus every nonzero $a \in A_k$ has a factorization

$$a = c_a \prod_{b \in B_k} b^{e_{a,b}}, \qquad e_{a,b} \geq 0,$$

where c_a depends only on $x_1, \ldots, x_{k-1}$ and is a nonzero constant times products of data passed to the lower-dimensional problem. Signs/units of these factorizations need not be stored because final signs use the original inputs. The baseline `__project_full(...,LAZARD,FACTOR)` provides this contract; `cad_tree` reverses the layer array to give $B_1, \ldots, B_n$.

MPP Theorem 5.1 states that a polynomial with positive last-variable degree and nonzero discriminant, leading coefficient and constant-in-the-last-variable coefficient is Lazard analytically delineable on a connected analytic submanifold where those three polynomials are valuation-invariant. Its moreover clause covers multiplication by x_k. Here "nonzero" means nonzero as a polynomial, not nonvanishing at every point of the cell. Nullifying lower-dimensional cells are therefore not excluded. MPP Definition 2.10 and Proposition 2.11 identify the continuous ordered root functions, their constant multiplicities, and valuation invariance on their sections and sectors.

To apply the theorem to B_k, let H be the product of its factors other than the possible associate of x_k. Irreducibility ensures that every factor of H has nonzero trailing coefficient, and squarefreeness and pairwise coprimality ensure that $\operatorname{disc}(H)$ is a nonzero polynomial. Its leading and trailing coefficients are products of those of the factors, while

$$\operatorname{disc}(H) = \pm \prod_b \operatorname{disc}(b) \prod_{b<c} \operatorname{res}(b, c)^2.$$

Consequently the projection hypotheses imply valuation invariance of these three polynomials on the parent cell. MPP Theorem 5.1 applies to H and, if needed, to $x_k H$. This is the product argument in MPP §5.3. If B_k is empty, there are no sections; if it consists only of an associate of x_k, its sole section is $x_k = 0$ and the claim is immediate. Distinct factors may have coincident specialized roots on lower-dimensional parents; we always use the distinct roots of their product, with no assumption that individual specialized lists are disjoint.

9.2 The meaning of stored nodes

The unique depth-zero cell is $\mathbb{R}^0$, with the empty point certificate. Suppose a parent node denotes a connected analytic semialgebraic cell S with sample $\alpha \in S$. The product theorem gives a fixed number m of ordered analytic functions $\theta_1 < \cdots < \theta_m : S \to \mathbb{R}$ whose values at every $u \in S$ are the distinct roots of the product of the Lazard evaluations of B_k. The jth section denotes

$$\Gamma_j = \{(u, \theta_j(u)) : u \in S\},$$

and the sectors denote

$$\Omega_j = \{(u,t) : u \in S,\ \theta_j(u) < t < \theta_{j+1}(u)\}, \quad 0 \le j \le m,$$

where $\theta_0 = -\infty$ and $\theta_{m+1} = +\infty$. With $m = 0$ this is the single cylinder $S \times \mathbb{R}$. The runtime root certificates describe only $(\alpha, \theta_j(\alpha))$; they are not asserted to be defining equations of the entire section.

The runtime root table is ordered by the distinct roots of the product. Stored boundaries retain the baseline `cad_root(b,r)` meaning: source polynomial index b in the current layer, and zero-based distinct-root index r within that source's Lazard evaluation at this parent. For the jth product root, set $sources_j$ to its nonempty source set and let

$$\nu_b(j) = \#\{h < j : b \in sources_h\}.$$

Choose the least $b \in sources_j$ for the stored reference $(b, \nu_b(j))$. Shared roots advance the counters of every source, so later roots of each source retain their correct ordinals. The `root` query, given the level, parent and reference, resolves this identity using the source lists, including valid alternative source identities for a shared root. The original `cad_tree` and its boundary type are not reinterpreted as local-union indices. The parent, source and per-source ordinal identify the same continuous section function guaranteed by projection, while the parent stack retains the merged sample root list.

These cells are semialgebraic: the prefix valuation r of the product is constant on S, so its Lazard root relation is the zero set, over S, of the rational polynomial $(\partial_1^{r_1} \cdots \partial_{k-1}^{r_{k-1}} H)/\prod r_i!$ (or the corresponding product including x_k). Choosing its jth distinct real root is a first-order real-algebraic condition. Closure of semialgebraic sets under projection yields the graph; this standard closure theorem is an explicit mathematical import, not an executable quantifier-elimination step. The sectors are semialgebraic by the same closure property.

10 Complete CAD: executable lifting and driver

10.1 Simplest rational samples in open intervals

We order reduced rationals first by increasing positive denominator and then by increasing absolute numerator, as in SageMath's `simplest_rational` specification. The following exact continued-fraction construction uses the integer-part/reciprocal method also described by `approxRational` in Haskell's `Data.Ratio` module, but with strictly open endpoints. These are implementation references, not new trusted mathematical oracles: SageMath interval documentation and Data.Ratio source. No floating-point operations are required. All integer parts use exact floor division of the QQ numerator by its

positive denominator.

Auxiliary procedure SAMPLE-RATIONAL. Inputs are open rational endpoints; a missing left or right endpoint denotes $-\infty$ or $+\infty$, respectively. Finite equal or reversed endpoints are rejected. Its implementation is `sample_rational` in the independent `rational_sample.hh` header.

if a, b are both finite **then**
    Reject unless $a < b$
    if $a < 0 < b$ **then**
        return 0
    end if
    if $b \le 0$ **then**
        return $-\text{POSITIVE}(-b, -a)$
    end if
    return $\text{POSITIVE}(a, b)$
else if $a = -\infty$ and $b = +\infty$ **then**
    return 0
else if $a = -\infty$ **then**
    if $b > 0$ **then**
        return 0
    end if
    return $\lceil b \rceil - 1$ ▷ $\lceil b \rceil = -\lfloor -b \rfloor$
else
    if $a < 0$ **then**
        return 0
    end if
    return $\lfloor a \rfloor + 1$
end if

Auxiliary procedure POSITIVE. Require finite $0 \le a < b$. Store integer parts in a stack K; this implements continued-fraction descent and recovery without recursive C++ calls.

$K \leftarrow [\,]$
loop
    $k \leftarrow \lfloor a \rfloor$; $r \leftarrow k + 1$
    if $r < b$ **then**
        break
    end if
    Push k onto K
    $f \leftarrow a - k$; $a \leftarrow 1/(b - k)$
    if $f = 0$ **then**
        $r \leftarrow \lfloor a \rfloor + 1$; **break** ▷ transformed upper endpoint is $+\infty$
    end if
    $b \leftarrow 1/f$
end loop
for each k in K in reverse order **do**
    $r \leftarrow k + 1/r$
end for
return r

Lemma 10.1 (Exact simplest open-interval sample)**.** *For every nonempty open interval with rational*

finite endpoints and possibly infinite outer endpoints, SAMPLE-RATIONAL terminates and returns its simplest rational. In particular the result belongs strictly to the interval.

Proof. For POSITIVE, put $k = \lfloor a \rfloor$. If $k + 1 < b$, the returned integer is strictly inside the interval. Otherwise $k \le a < b \le k + 1$, and $x \mapsto 1/(x - k)$ is a bijection from (a, b) onto $(1/(b - k), 1/(a - k))$, with inverse $y \mapsto k + 1/y$. If $a = k$, the latter upper endpoint is $+\infty$; the code immediately chooses an integer there and never computes $1/0$. The stack records the inverse maps, whose composition maps the current interval onto the original positive interval. Reverse recovery therefore preserves strict membership, including integer endpoints that must be excluded.

We prove the stronger statement that the positive result p_0/q_0 has both $p_0 \le p$ and $q_0 \le q$ for every reduced positive candidate p/q. In an integer base case, every candidate satisfies $p > kq$, hence $p \ge kq + 1 \ge k + 1$, and $q \ge 1$. For a nonterminal step, a reduced candidate $x = p/q$ transforms to the reduced $y = q/(p - kq)$, with both parts positive. If the chosen transformed result is u/v, induction gives $u \le q$ and $v \le p - kq$. The recovered result is the reduced fraction $(ku + v)/u$: its denominator is at most q, and its numerator is at most $kq + (p - kq) = p$, since $k \ge 0$. Thus recovery preserves simultaneous minimality and in particular the specified denominator-first order. Reflection preserves absolute numerators on negative intervals. Zero is simplest whenever admitted. On an unbounded interval, the specified integer is the admitted integer of least absolute value, so it too is simplest.

For termination, write the finite current endpoints in reduced form as $a = p/q$ and $b = s/t$. In a nonterminal step with noninteger a, the new denominators are $s - kt$ and $p - kq$, with $0 < s - kt \le t$ and $0 < p - kq < q$. Each is coprime to its corresponding numerator, and their sum is strictly less than $q + t$. An integer lower endpoint instead takes the immediate unbounded base case. The positive integer measure $q + t$ therefore decreases until a base case is reached. The finite stack is then recovered using positive divisors. □

For algebraic CAD boundaries, this lemma applies to the certified QQ inner gap, not directly to the irrational endpoints. It guarantees simplestness in that inner gap, not necessarily in the entire algebraic sector. For adjacent roots $\alpha < \beta$, separation gives rational a, b with $\alpha < a < b < \beta$; hence $\alpha < a < q < b < \beta$. Outer root brackets similarly give valid one-sided gaps. Section certificates are unchanged. Sample selection imports no algebraic-coefficient operations.

Algorithm 12 LIFT-STACK: one parent and its complete children

Require: Parent sample A of S at depth $k-1$; current variable x_k; ordered results $G = [(p_b, v_b)]_{b \in B_k}$ of LAZARD-QQ on A

Ensure: $2m+1$ ordered children, valid samples, dimensions, root sources and a stack record

$p \leftarrow \prod_{b \in B_k} p_b$ in QQ polynomials ▷ empty product is 1
$R \leftarrow \text{ROOTS-QQ}(p, A, x_k)$ ▷ sorted, distinct, complete root certificates
Set per-source counters $N_b \leftarrow 0$
for each $D_j \in R$ in increasing order **do**
    $sources_j \leftarrow \{b : \text{POINT-ZERO}(p_b, D_j)\}$; require $sources_j \neq \varnothing$
    $b \leftarrow \min(sources_j)$; $boundary_j \leftarrow (b, N_b)$
    for $c \in sources_j$ **do**
        $N_c \leftarrow N_c + 1$
    end for
end for
$m \leftarrow |R|$; $d \leftarrow \dim(S)$ ▷ $d = 0$ for the depth-zero cell
for $i = 0, \ldots, m$ **do**
    if $m = 0$ **then**
        $q \leftarrow 0$
    else if $i = 0$ **then**
        $q \leftarrow \text{SAMPLE-RATIONAL}(-\infty, \inf(\text{last box of } D_1))$
    else if $i = m$ **then**
        $q \leftarrow \text{SAMPLE-RATIONAL}(\sup(\text{last box of } D_m), +\infty)$
    else
        Refine D_i, D_{i+1} until their last boxes are strictly separated
        $q \leftarrow \text{SAMPLE-RATIONAL}(\sup(\text{last box of } D_i), \inf(\text{last box of } D_{i+1}))$
    end if
    Append Sector i to this parent's child block, with adjacent source/root boundary references
    Store sample $\text{APPEND-RATIONAL}(A, x_k, q)$ and dimension $d+1$
    if $i < m$ **then**
        Append Section $i+1$, sample D_{i+1}, reference $boundary_{i+1}$ and dimension d
    end if
end for
Store R, G and *sources* in the parent-indexed stack table

APPEND-RATIONAL appends the QQ equation $x_k - q = 0$, the strict interval $[q-1, q+1]$, and a rational shift placing it in the positive half-line. Its derivative is 1 and its unique root is q; existing prefix certificates are retained. This proves validity and termination of this elementary helper.

Algorithm 13 FULL-CAD: exact rational-data projection and lifting

Require: Finite QQ input list F; projection variable order V
Ensure: A finite sign-invariant CAD of $\mathbb{R}^{|V|}$, its samples, dimensions and original-input leaf signs

Reject duplicate variables, unbound variables and negative exponents
Retain original QQ inputs; form denominator-cleared ZZ inputs Z
$P \leftarrow \text{PROJECT-FULL}(Z, V, \text{LAZARD}, \text{FACTOR})$
Own the projection comparator; rebind returned polynomial comparators
Initialize the tree; reverse V, P into lifting order $x_1, \ldots, x_n$ and $B_1, \ldots, B_n$
for $k = 1, \ldots, n$ **do**
    Parents are the preceding level in storage order, or the one virtual empty cell for $k = 1$
    for each parent S with sample A **do**
        $G \leftarrow [\text{LAZARD-QQ}(b, (x_1, \ldots, x_k), A) : b \in B_k]$
        $\text{LIFT-STACK}(S, A, x_k, G)$
    end for
end for
for each leaf, or the one empty leaf if $n = 0$ **do**
    Copy its sample to A; store $[\text{POINT-SIGN}(f, A) : f \in F]$
end for
return the tree, samples, dimensions, stacks and signs

10.2 Loop invariants and implementation correspondence

For LIFT-STACK, after i iterations, the child block contains exactly the first i sectors and first i sections (except that the final iteration adds only the last sector). Every stored sample belongs to its child; boundary references preserve their source polynomial index and per-source distinct-root ordinal; dimensions equal the parent dimension plus 0 or 1. Refining a certificate changes neither its selected point nor its ordinal. The source loop maintains N_b equal to the number of roots already seen for source b. Incrementing all sources, including secondary ones at shared roots, proves that every stored reference has the baseline meaning. An empty product has no roots and yields one sector, not an error.

For FULL-CAD, before level k, all prior levels partition their coordinate spaces, have connected analytic semialgebraic cells and valid samples, and the immediately preceding level is valuation-invariant for the nonzero members of A_{k-1}. During a level, exactly the already visited parents have complete, contiguous child blocks; each earlier child block stays unchanged as the next is appended. Thus CSR parent/child indices, parallel sample and dimension arrays, and the stack array indexed by parent remain aligned. The root-level virtual parent is encoded by `SIZE_MAX`, not by an ordinary node index. The sign loop has filled exactly its visited leaf rows, each in original input order.

The storage type is `full_cad_result`, declared separately in `full_cad_result.hh`. It only stores nodes and associated certificates; its append operations keep the parallel tables aligned. The corresponding algorithm code is `__lift_stack` and `full_cad`. Algebraic extensions use ROOTS-QQ; sector samples are chosen by `sample_rational` and certified by `append_rational`. Neither loop constructs an algebraic coefficient. The Lazard result value type is declared separately in `lazard_qq_result.hh`; its data consumer need not include the Lazard evaluation algorithm header. The CAD result owns its comparator through a shared pointer; projection polynomials, sample chains and root stacks borrow it. Copies of the whole result retain this ownership. A detached point copy requires the CAD comparator to remain alive. Lazard representatives separately retain shared ownership of their own comparator.

10.3 Correctness and termination

Theorem 10.2 (Complete rational-data CAD)**.** *Relative to the stated trusted contracts for projection, factorization and tree storage, the imported MPP theorem and the preceding real-point algorithms, FULL-CAD terminates and returns a finite cylindrical algebraic decomposition covering all of* $\mathbb{R}^n$*. Its cells are nonempty, connected, semialgebraic and pairwise disjoint, its dimensions and samples are correct, and each recorded input sign is constant over the corresponding leaf.*

Proof. Start with $\mathbb{R}^0$, its single empty point and dimension 0. Its polynomial data are constants and thus valuation-invariant. Suppose the driver invariant holds before level k. The projection contract makes all required data for B_k valuation-invariant on each parent S. The product argument in Section 9 gives analytic ordered root functions and constant prefix valuations on S. LAZARD-QQ correctly represents their specialization at the sample using QQ polynomials. Specialization is a ring homomorphism, so the product of representatives specializes to the product of their nonzero Lazard values and is nonzero. ROOTS-QQ therefore applies and returns precisely their m distinct sample roots. This identifies all sections by order. Every such root belongs to some factor's Lazard value, which proves that the computed source list is nonempty. POINT-ZERO certifies each recorded source at that sample.

Sections and sectors partition every fibre over S: the m equalities and $m+1$ strict intervals exhaust the real line without intersection. Sections are analytic graphs over S. Each sector is an open analytic submanifold of $S\times\mathbb{R}$, homeomorphic to $S\times(0,1)$ using its continuous boundary functions (with the usual one-sided or unbounded parametrization at infinity). They are connected and nonempty, of dimensions $\dim S$ and $\dim S+1$, respectively; their semialgebraicity was shown above. At the sample fibre the strict rational brackets and Lemma 10.1 provide rational numbers inside every sector, and the root certificates provide section samples. APPEND-RATIONAL certifies the former. Different parents are disjoint; each child's projection is exactly its parent. Together these facts establish coverage, disjointness and cylindricality at the new level as well as the child-block invariant.

By MPP Proposition 2.11 the product of B_k is valuation-invariant on each child. MPP Proposition 3.4 then gives valuation invariance of every individual factor on each connected child. Lower-variable contents inherit their parent's valuation with a trailing zero coordinate. Multiplicativity of valuation and the factorization of each member of A_k prove the driver's valuation invariant for the new level. This also handles constants, omitted units and repeated factors. The induction therefore continues through level n; it does not rely merely on sign invariance of the parent samples.

A nonzero polynomial has valuation zero at a point exactly when its ordinary value is nonzero. On a connected valuation-invariant cell it is consequently identically zero, or nowhere zero. In the latter case continuity makes its sign constant. Clearing positive denominators preserves the sign of each original QQ input; zero inputs have sign zero everywhere. POINT-SIGN evaluates the original input at its certified sample, and hence stores the sign of the whole leaf. For $n=0$ this is rational constant evaluation at the sole empty leaf.

Projection is a terminating trusted operation. At each level there are finitely many parents and projection factors. LAZARD-QQ and ROOTS-QQ terminate by their proved contracts; each finite root list yields $2m+1$ children. Separation of two distinct roots terminates by REFINE, and all source/sign queries terminate by POINT-ZERO/SIGN. SAMPLE-RATIONAL terminates by Lemma 10.1. Induction over the finite number of levels and then the finite leaf list proves termination of the driver and all its subloops. □

This theorem does not give a complexity bound or a machine-checked C++ refinement. In particular, it imports the baseline algorithms' specified behavior rather than claiming that wrapper checks prove their internals.

11 Examples and mandatory regressions

1. $T = [x^2 - 2]$, $B = [1, 2]$, $f = (x^2 - 2)y$. PREFIX-ZERO is true at order 0 and false for $2xy$. Return representative $2xy$ and valuation (1). The meaning is $2\sqrt{2}y$; no such coefficient is stored.

2. With the same T, B, $f = (x^2 - 2)^2$ gives $r = 2$ and representative $(12x^2 - 8)/2 = 6x^2 - 4$. Its value is 8. Reducing f modulo T before these derivatives would incorrectly erase the entire input.

3. $T = [x^2 - 2, z^2 - 2]$, positive x and negative z. POINT-ZERO on $z - x$ returns false, on $z + x$ returns true. Norm roots and coordinate-interval overlap alone cannot distinguish the branches.

4. $f = xz - y$ at $(x, y) = (0, 0)$, order x, y, z, returns -1 and $(0, 1)$. Do not specialize all rational coordinates in advance.

5. $T = [(x - 1)(x - 2)]$, a box around 1, and $c = x - 1$. The representative and its quotient-ring residue may be nonzero although POINT-ZERO must return true. No quotient-field assumption is permitted.

6. $T = [(x - 1)(x - 2)]$, selected $x = 1$, $p = (x - 2)y$. ROOTS-QQ must remove the $x = 2$ degree-zero branch before appending the equation. Otherwise the whole line $x = 2$ would violate the finite-solution hypothesis.

7. $T = [(x-1)^2]$ is regular but not squarefree. RCtoSqrfree must remove this multiplicity even when the chain has length one.

8. Root isolation must include negative roots, zero roots, rational roots hit by bisection, close roots, and specializations with degree drop or repeated roots.

12 Implementation audit and formal-proof boundary

Review is against the numbered algorithms in this PDF, not against passing tests. Each function must have a traceability entry: algorithm/line, concrete types, preconditions, invariant, decreasing measure or convergence lemma, and regression. A different backend is a failed conformance check until the specification and proof are changed explicitly. No implicit change of mathematical representation is allowed.

Obligation	Implementation	Implementation contract
Exact QQ intervals	`rational_interval.hh`	Independent arithmetic and set operations; no root or polynomial dependency. Exact endpoint operations with explicit domain checks.
Triangular algorithms	`regular_chain.hh`, backed by `charset.hh`	XY16 WRSD and squarefree decomposition. The multiplicity scan includes the first level. Chains borrow external comparators and validate contiguous prefix order.
XZ-LIFT / NSHR / REFINE	`real_algebraic.hh`	Rational bounds, positive shifts and strict root certificates retained; bottom-up refinement follows the numbered pseudocode.

Obligation	Implementation	Implementation contract
POINT-ZERO / SIGN	`point_zero`, `point_sign`	WRSD set labels and interval elimination at the original selected point; no field quotient or norm-root shortcut.
PREFIX-ZERO / LAZARD-QQ	`lazard_qq.hh`	Active-monomial grouping; frozen QQ representatives; ordered derivatives and factorial division; no irrational-coefficient object.
REENCODE / ROOTS-QQ	`real_algebraic.hh`	Membership plus certified box comparison; WRSD degree branch selection before squarefree isolation; exact duplicate removal.
Full lifting	`full_cad.hh`	Free lifting and driver functions use certified real points, trusted projection and the storage interface of `full_cad_result`.

For the first coordinate, the implementation invokes XZ-LIFT with an empty prefix. Then $L = U$ is an exact rational polynomial, so this is UNI preceded by a rational translation and followed by its inverse; it implements the first step of ISOLATE-REGULAR with the same root set and additional explicit certificates. REENCODE additionally checks point membership before box matching; this is a sound strengthening of its precondition, not an interval-overlap shortcut. Regression results are recorded separately in the current module test report. This is regression evidence; inherited triangular routines, UNI and the projection operator still need implementation refinement proofs. The audit does not claim author-Maple cross-validation.

For formalization the dependency graph is acyclic:

$$\begin{array}{c} \text{rational arithmetic, polynomials, interval inclusion} \\ \Downarrow \\ \text{XY16 decomposition and XZ06 certified refinement} \\ \Downarrow \\ \text{POINT-ZERO} \longrightarrow \text{PREFIX-ZERO} \longrightarrow \text{LAZARD-QQ.} \end{array}$$

ROOTS-QQ additionally uses squarefree decomposition and NREALZERO. The proofs above assume correctness of the explicitly cited algorithms, exact rational primitives and univariate isolation. Formal verification must discharge these imports and the C++ representation refinements. The MPP projection/delineability theorem is a named import; Theorem 10.2 proves lifting and the driver relative to it. Machine refinement of the C++ implementation remains a separate obligation. Current tests and manual conformance review do not establish machine-checked correctness or large-instance performance guarantees.

References

[1] S. McCallum, A. Parusiński and L. Paunescu, Validity proof of Lazard's method for CAD construction, *Journal of Symbolic Computation* 92 (2019), 52–69. Version checked: `https://arxiv.org/abs/1607.00264v2`, Algorithm 2.12, Propositions 2.11 and 3.4, Theorem 5.1 and Section 5.3. Already cited by the trusted baseline project research.

[2] B. Xia and L. Yang, *Automated Inequality Proving and Discovering*, World Scientific, 2016. Chapters 2–3. Local source: `docs/references/triangular-set/`, file beginning "Xia [and] Yang - 2016".

[3] B. Xia and T. Zhang, Real Solution Isolation Using Interval Arithmetic, *Computers and Mathematics with Applications* 52 (2006), 853–860. DOI: 10.1016/j.camwa.2006.06.003. Local source: `Xia-Zhang-2006-Real-Solution-Isolation-Using-Interval-Arithmetic.pdf`.

[4] B. Xia and L. Yang, An Algorithm for Isolating the Real Solutions of Semi-algebraic Systems, *Journal of Symbolic Computation* 34 (2002), 461–477. Supplementary reference for regularization; its old isolation procedure is not the completeness basis used here.

13 Checked interfaces and lexicographic implementation

The public polynomial entry points check variable coverage, nonnegative exponents and coordinate compatibility before conversion. Their internal `__` variants consume normalized QQ or ZZ polynomials in the specified lexicographic order and use assertions for caller obligations. Data types retain representation, ownership and storage invariants; decomposition, refinement and CAD construction remain free algorithms.

For PREFIX-ZERO the implementation puts every active variable before the frozen variables, preserving the latter's relative order. Equal active monomials are therefore contiguous. Collecting each contiguous group produces exactly the same coefficient polynomial as the abstract map in Algorithm 7; deleting the common active factor preserves its sorted unique frozen terms. No second sort or map is needed. LAZARD-QQ calls this ordered core on its unreduced QQ representative and uses the exact point-zero core on the complete coefficient.

XZ-LIFT uses ordered interval evaluation for its leading-coefficient and root-bound calculations. After the two bound polynomials pass the squarefree test it calls the asserted UNI core, avoiding duplicate release-mode gcd computations. These changes preserve all acceptance predicates and refinement evidence above.

The CAD wrapper supplies the checked ZZ projection inputs and original QQ sign inputs to the lexicographic driver. LIFT-STACK is an internal free algorithm. Lazard representatives share an immutable owned comparator with the result, and root-source tests use coefficients converted once to that order. The stored root identities and cell semantics are unchanged.